Creating CI/CD pipelines in AWS can be done using AWS CodePipeline, which is a continuous integration and continuous delivery service for fast and reliable application and infrastructure updates. Here's a step-by-step guide to create a CI/CD pipeline in AWS:

Step 1: Set Up Your Environment

1. **Create an AWS Account**: If you don't have an AWS account, [create one here](https://aws.amazon.com/).

2. **IAM User**: Create an IAM user with the necessary permissions to access AWS CodePipeline, CodeBuild, and other relevant services. Attach the following policies to your user:
   - `AWSCodePipelineFullAccess`
   - `AWSCodeBuildDeveloperAccess`
   - `AmazonS3FullAccess`
   - `AWSCodeCommitFullAccess` (if using CodeCommit for source control)

Step 2: Create a Code Repository

You can use AWS CodeCommit, GitHub, Bitbucket, or another repository. For simplicity, we'll use CodeCommit.

1. **Create a Repository**:
   - Go to the CodeCommit console.
   - Click "Create repository."
   - Name your repository and provide a description.
   - Click "Create."

Step 3: Create a Build Project in CodeBuild

1. **Go to AWS CodeBuild Console**:
   - Click "Create build project."
   - Name your project.

2. **Source**:

   - Choose your repository provider (e.g., AWS CodeCommit, GitHub).

   - Select your repository and branch.

3. **Environment**:

   - Choose the environment image (managed image is recommended).

   - Select the operating system, runtime, and version.

   - Choose the build specifications (buildspec.yml file or insert commands directly).

4. **Buildspec File**:

   - Create a `buildspec.yml` file in the root of your repository. This file defines the build commands and settings.

   ```yaml
   version: 0.2

   phases:
     install:
       runtime-versions:
         nodejs: 12
     pre_build:
       commands:
         - echo Installing source NPM dependencies...
         - npm install
     build:
       commands:
         - echo Build started on `date`
         - npm run build
   artifacts:
     files:
       - '**/*'
   ```

```

5. **Artifacts**:
   - Define the output artifacts (e.g., S3 bucket).

6. **Create the Build Project**.

### Step 4: Create the Pipeline in CodePipeline

1. **Go to AWS CodePipeline Console**:
   - Click "Create pipeline."
   - Name your pipeline.

2. **Source Stage**:
   - Select your source provider (e.g., CodeCommit).
   - Select your repository and branch.

3. **Build Stage**:
   - Select AWS CodeBuild.
   - Choose the build project created earlier.

4. **Deploy Stage**:
   - Add a deploy stage to deploy your application. Common options include:
     - AWS Elastic Beanstalk
     - AWS ECS
     - AWS Lambda
     - S3 (for static websites)

   Example for S3:
   - Choose "Deploy" action provider as Amazon S3.
```

- Choose your target S3 bucket.

5. **Review and Create**:
   - Review the stages.
   - Click "Create pipeline."

Step 5: Trigger the Pipeline

1. **Push Code**:
   - Push code to your repository. This should trigger the pipeline automatically.

2. **Monitor Pipeline**:
   - Go to the CodePipeline console to monitor the execution of your pipeline stages.

Example CI/CD Workflow

Here's an example of what your workflow might look like:

1. **Developer Pushes Code**:
   - Code is pushed to CodeCommit.

2. **CodePipeline Triggers**:
   - The source stage picks up the changes.

3. **Build Stage**:
   - CodeBuild builds the application using the `buildspec.yml` file.

4. **Deploy Stage**:
   - CodePipeline deploys the build artifacts to the specified AWS service (e.g., S3, Elastic Beanstalk).

By following these steps, you can set up a basic CI/CD pipeline in AWS. You can customize and extend this pipeline based on your specific requirements and integrate additional services like AWS CodeDeploy, AWS Lambda, and more.